Softmax regression

SPRING 2023

ZORAN TIGANJ

Softmax Regression

- The Logistic Regression model can be generalized to support multiple classes directly, without having to train and combine multiple binary classifiers.
- This is called Softmax Regression, or Multinomial Logistic Regression.
- The idea is simple: when given an instance x, the Softmax Regression model first computes a score $s_k(x)$ for each class k, then estimates the probability of each class by applying the softmax function (also called the normalized exponential) to the scores.
- The equation to compute $s_k(x)$ is just like the equation for Linear Regression prediction

$$s_k(\mathbf{x}) = \mathbf{x}^\mathsf{T} \boldsymbol{\theta}^{(k)}$$

- Note that each class has its own dedicated parameter vector $\boldsymbol{\theta}^{(k)}$. All these vectors are typically stored as rows in a parameter matrix $\boldsymbol{\Theta}$.

Softmax Regression

- Once you have computed the score of every class for the instance **x**, you can estimate the probability $\hat{p}$ that the instance belongs to class k by running the scores through the softmax function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp\left(s_k\left(\mathbf{x}\right)\right)}{\sum_{j=1}^{K} \exp\left(s_j\left(\mathbf{x}\right)\right)}$$

In this equation:

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is a vector containing the scores of each class for the instance $\mathbf{x}$.
- $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance $\mathbf{x}$ belongs to class k, given the scores of each class for that instance.

Softmax Regression

- Once you have computed the score of every class for the instance **x**, you can estimate the probability $\hat{p}$ that the instance belongs to class k by running the scores through the softmax function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp\left(s_k\left(\mathbf{x}\right)\right)}{\sum_{j=1}^{K} \exp\left(s_j\left(\mathbf{x}\right)\right)}$$

- The function computes the exponential of every score, then normalizes them (dividing by the sum of all the exponentials).
- The scores are generally called logits or log-odds

Softmax Regression

- Just like the Logistic Regression classifier, the Softmax Regression classifier predicts the class with the highest estimated probability (which is simply the class with the highest score)

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ \sigma(\mathbf{s}(\mathbf{x}))_k = \underset{k}{\operatorname{argmax}}\ s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \left(\left(\boldsymbol{\theta}^{(k)} \right)^{\mathsf{T}} \mathbf{x} \right)$$

The *argmax* operator returns the value of a variable that maximizes a function. In this equation, it returns the value of k that maximizes the estimated probability $\sigma(\mathbf{s}(\mathbf{x}))_k$.

- The Softmax Regression classifier predicts only one class at a time (i.e., it is multiclass, not multioutput), so it should be used only with mutually exclusive classes, such as different types of plants. You cannot use it to recognize multiple people in one picture.

Cross entropy

- Now that we know how the model estimates probabilities and makes predictions, let's take a look at training.
- The objective is to have a model that estimates a high probability for the target class (and consequently a low probability for the other classes).
- Likelihood function for one data point is: $L = \Pi_{k=1}^{K} {p_k}^{y_k}$
- Minimizing the cost function (negative log likelihood), called the cross entropy, should lead to this objective because it penalizes the model when it estimates a low probability for a target class.

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

In this equation:

- $y_k^{(i)}$ is the target probability that the ith instance belongs to class k. In general, it is either equal to 1 or 0, depending on whether the instance belongs to the class or not.

A little more about Cross entropy

- Cross entropy originated from information theory.
- Suppose you want to efficiently transmit information about the weather every day. If there are eight options (sunny, rainy, etc.), you could encode each option using three bits because $2^3 = 8$.
- However, if you think it will be sunny almost every day, it would be much more efficient to code "sunny" on just one bit (0) and the other seven options on four bits (starting with a 1).
- Cross entropy measures the average number of bits you actually send per option.
  - If your assumption about the weather is perfect, cross entropy will be equal to the entropy of the weather itself (i.e., its intrinsic unpredictability).
  - But if your assumptions are wrong (e.g., if it rains often), cross entropy will be greater by an amount called the Kullback Leibler (KL) divergence.
  - The cross entropy between two probability distributions p and q is defined as

$$H(p,q) = -\sum_x p(x) \log q(x)$$

Cross entropy

- We can now compute the gradient of the cost function $J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$

$$\nabla_{\boldsymbol{\theta}^{(k)}} J\left(\mathbf{\Theta}\right) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)}$$

- Then we can use Gradient Descent (or any other optimization algorithm) to find the parameter matrix $\mathbf{\Theta}$ that minimizes the cost function.

Example of Softmax Regression

- Let's use Softmax Regression to classify the iris flowers into all three classes.
- Scikit-Learn's LogisticRegression uses one-versus-the-rest by default when you train it on more than two classes, but you can set the multi_class hyperparameter to "multinomial" to switch it to Softmax Regression.
- You must also specify a solver that supports Softmax Regression, such as the "lbfgs" solver.
- It also applies ℓ_2 regularization by default, which you can control using the hyperparameter C:

```
X = iris["data"][:, (2, 3)]  # petal length, petal width
y = iris["target"]

softmax_reg = LogisticRegression(multi_class="multinomial",solver="lbfgs",
C=10)
softmax_reg.fit(X, y)
```

Example of Softmax Regression

- So the next time you find an iris with petals that are 5 cm long and 2 cm wide, you can ask your model to tell you w hat type of iris it is.

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]])
array([[6.38014896e-07, 5.74929995e-02, 9.42506362e-01]])
```

- The model answered Iris virginica (class 2) with 94.2% probability and Iris versicolor with 5.8% probability:

Example of Softmax Regression

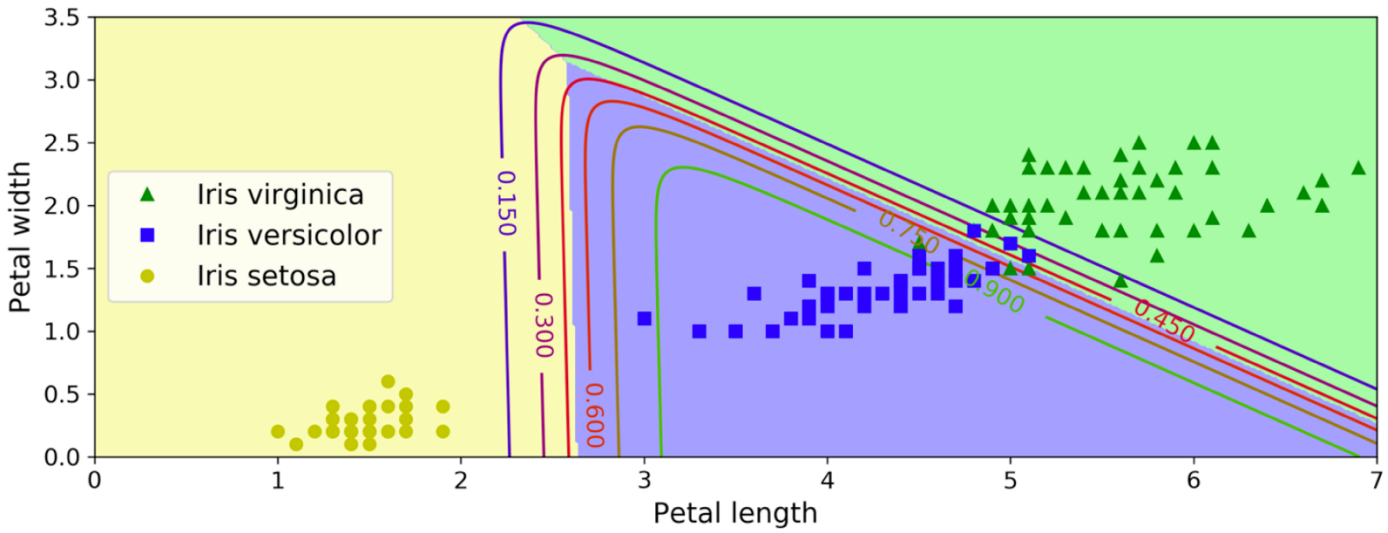

Figure 4-25. Softmax Regression decision boundaries

- Notice that the decision boundaries between any two classes are linear.
- The figure also shows the probabilities for the Iris versicolor class, represented by the curved lines.
- Notice that the model can predict a class that has an estimated probability below 50%.
- or example, at the point where all decision boundaries meet, all classes have an equal estimated probability of 33%.

Next time

- Support Vector Machines (SVM), from Chapter 5 from *Hands on machine learning textbook*